



Sales Forecasting and Optimization

“Amazon Sales Dataset”



1.Data collection and Exploration

Data Collection

For this project, we utilized a publicly available dataset sourced from [Kaggle , "*thedevastator/unlock-profits-with-e-commerce-sales-data*"]. The dataset used is titled [Amazon Sale Report.csv], and it contains [23 columns , 128975 rows]. The dataset was chosen due to its relevance to the implementation of machine learning models, particularly ensemble methods such as the Random Forest algorithm.

EDA

To initiate the data preprocessing workflow, we began by examining the structure and general characteristics of the dataset using the `.info()` method:

This command provided a concise summary of the dataset, including the number of non-null entries, data types of each feature, and memory usage. It served as a foundational step for understanding the data's dimensionality and identifying any potential issues related to missing values or incorrect data types.

Subsequently, I utilized the `convert_dtypes()` method to ensure that all columns were automatically cast to the most appropriate and optimized data types:

This approach facilitated more efficient memory usage and improved compatibility with downstream operations, particularly when working with mixed data types

The dataset included a column named 'Date', which was originally in string format. To enable time-based analysis, I converted this column into a proper datetime format using the `pd.to_datetime()`.

Here, I specified the format explicitly as `%m-%d-%y` to match the input format and set the `dayfirst=True` parameter to ensure accurate parsing in cases of potential ambiguity. This transformation was essential for extracting temporal patterns, trends, or seasonality in subsequent analyses.

To better understand the structure and variability within the dataset, I computed the **number of unique values** across all columns:

This allowed me to identify categorical versus continuous features and evaluate whether certain columns might be redundant or suitable for encoding.

In parallel, I applied the `pd.unique()` function to extract all distinct values present in each column:

This step was particularly useful for exploring the categorical features and detecting any inconsistencies or anomalies such as typos or formatting issues in string-based fields.

One of the key challenges in the dataset was the presence of missing values in the 'Courier Status' column. To address this, I investigated the relationship between the 'Status' and 'Courier Status' columns by extracting all unique combinations of these two features.

This exploratory step enabled me to assess whether the value of 'Courier Status' could be inferred based on the corresponding 'Status'. If a strong, deterministic mapping existed between these two variables, it provided a reliable basis for imputing the missing 'Courier Status' values in a consistent and data-driven manner.

To gain a deeper understanding of the underlying structure of the data, I conducted an Exploratory Data Analysis (EDA) using Python-based tools, primarily **Pandas**, **Matplotlib**, and **Seaborn**. These libraries were chosen for their robust capabilities in data manipulation and visualization.

Key trends and observations identified during the EDA process include:

- **Missing Values:** we checked for missing or null values using `isnull().sum()` and visualized them using heatmaps (via Seaborn). Handling of missing data was performed accordingly—either through imputation or by removing the affected records, depending on the proportion and significance of the missingness.
- **Feature Distributions:** we examined the distribution of continuous and categorical variables using histograms and count plots. This helped identify skewness, outliers, and potential need for normalization.
- **Correlation Analysis:** we employed correlation heatmaps to identify multicollinearity and strong linear relationships between independent variables and the target variable. This was essential for feature selection and understanding variable importance.

Tools

- **Pandas:** Used for data loading, cleaning, and transformation due to its efficient DataFrame structure and comprehensive functionality.
- **Matplotlib & Seaborn:** Employed for visualizing distributions, trends, correlations, and relationships. Seaborn was particularly useful for producing aesthetically pleasing and informative plots with minimal code.

- **NumPy:** Utilized for numerical computations and matrix operations when manipulating the data prior to model training.

2.Data Analysis and Visualization

Data Cleaning

Following the initial exploration of the dataset, we undertook a structured data cleaning process to handle missing values, remove irrelevant columns, and ensure the integrity of key features for model training.

Standardizing Missing Values

To maintain consistency in how missing data is represented across the dataset, we replaced placeholder strings such as "<NA>" with the standard NumPy representation of missing values:

This step was critical for ensuring compatibility with pandas-based functions that are designed to detect and manipulate missing entries (`np.nan`).

Dropping Columns with High Missingness or Irrelevance

Certain columns were identified as either having an excessive proportion of missing values or lacking meaningful variance. Specifically:

['Unnamed: 22', 'fulfilled-by', 'promotion-ids']

These columns were eliminated as they contributed little to no informational value or were not reliably populated across the dataset.

Additionally, the 'currency' column was removed due to its singular unique value ("INR"), which offered no discriminative power for the analysis.

Removing Rows with Incomplete Shipping Information

Given that only a small fraction of records contained missing values in critical shipping-related fields, I opted to remove these rows entirely

This decision was based on the rationale that imputing detailed geographic data could introduce bias or inaccuracy, whereas the impact of removing a small number of rows on overall analysis would be negligible.

The 'Amount' column, which contains numerical data related to transaction value, had missing entries. I addressed this by imputing the mean of the column:

Mean imputation is a widely used technique when the data is approximately normally distributed and the proportion of missing values is low. It preserves the central tendency without drastically altering the distribution.

Context-Aware Imputation of 'Courier Status'

Given the presence of missing values in the 'Courier Status' field, I employed a conditional imputation strategy based on the associated 'Status' field:

- For rows where 'Status' is "Cancelled", I identified the most frequently occurring 'Courier Status' value and used it to fill the missing entries.
- For cases where 'Status' indicated successful shipment or return (i.e., "Shipped - Returned to Seller" or "Shipped - Delivered to Buyer"), I filled the missing 'Courier Status' with the label "Shipped".

This method of imputing categorical values based on logical associations ensured that the imputation remained context-aware and domain-relevant, minimizing potential distortions in model training.

Exporting the Cleaned Dataset

Upon completion of the data cleaning and preprocessing procedures, I exported the refined dataset to a CSV file to preserve the transformed version for subsequent analysis and modeling tasks:

By specifying `index=False`, I ensured that the DataFrame's index was not included in the exported file, as it did not contain meaningful information in this context. Saving the cleaned dataset facilitates reproducibility, allows for efficient version control, and enables downstream processes—such as feature engineering, model training, or exploratory visualization—to proceed without reapplying the data cleaning steps.

This file serves as the standardized, preprocessed dataset for the next stages of the machine learning pipeline.

Analysis

Descriptive Statistics and Exploratory Data Analysis (EDA)

To begin the exploratory analysis, I generated descriptive statistics for the categorical variables using the `.describe()` function with the `include='O'` parameter.

This provided insights into the count, unique values, top occurrences, and frequency for each object-type column, allowing me to identify the dominant categories and potential anomalies.

Daily Sales Trends

I extracted the daily aggregated sales by grouping the dataset by the 'Date' column and computing the total quantity (Qty) and sales amount (Amount) for each day. This aggregation facilitated a temporal analysis of purchase behavior.

Top Sales Days

To identify the most profitable days, I sorted the daily_sales DataFrame in descending order by total_amount and extracted the top 10 dates.

This enabled a targeted understanding of peak sales periods, which could correspond to promotions, campaigns, or seasonal patterns.

Monthly Sales Performance

For a broader view of temporal patterns, I extracted the month name from the 'Date' column and grouped the data by month.

Since the data spanned April to June, I manually ordered the months for chronological coherence:

This monthly breakdown supported the identification of seasonal variations in sales volume and revenue.

Correlation Between Quantity and Amount

To assess the linear relationship between the number of units sold and the total transaction value, I calculated the Pearson correlation coefficient:

The heatmap visually represented the strength of association, confirming whether higher quantities typically correspond to higher revenue.

Sales Analysis by Product Attributes

By Size

Sales were analyzed by item size to determine which sizes generated the highest revenue.

By Category

Similarly, I aggregated sales by product category.

These aggregations provided actionable insights into product performance by size and category.

Order Status Distribution and Return Rates

I examined the distribution of order statuses through the count of status column.

To analyze customer return behavior, I filtered the dataset for statuses indicative of returns.

The return rate was then computed by category as a percentage of total sales amount.

$$[\text{Returned.sum} / \text{Original.sum}] * 100$$

This allowed for identifying which product categories had disproportionately high return or cancellation rates.

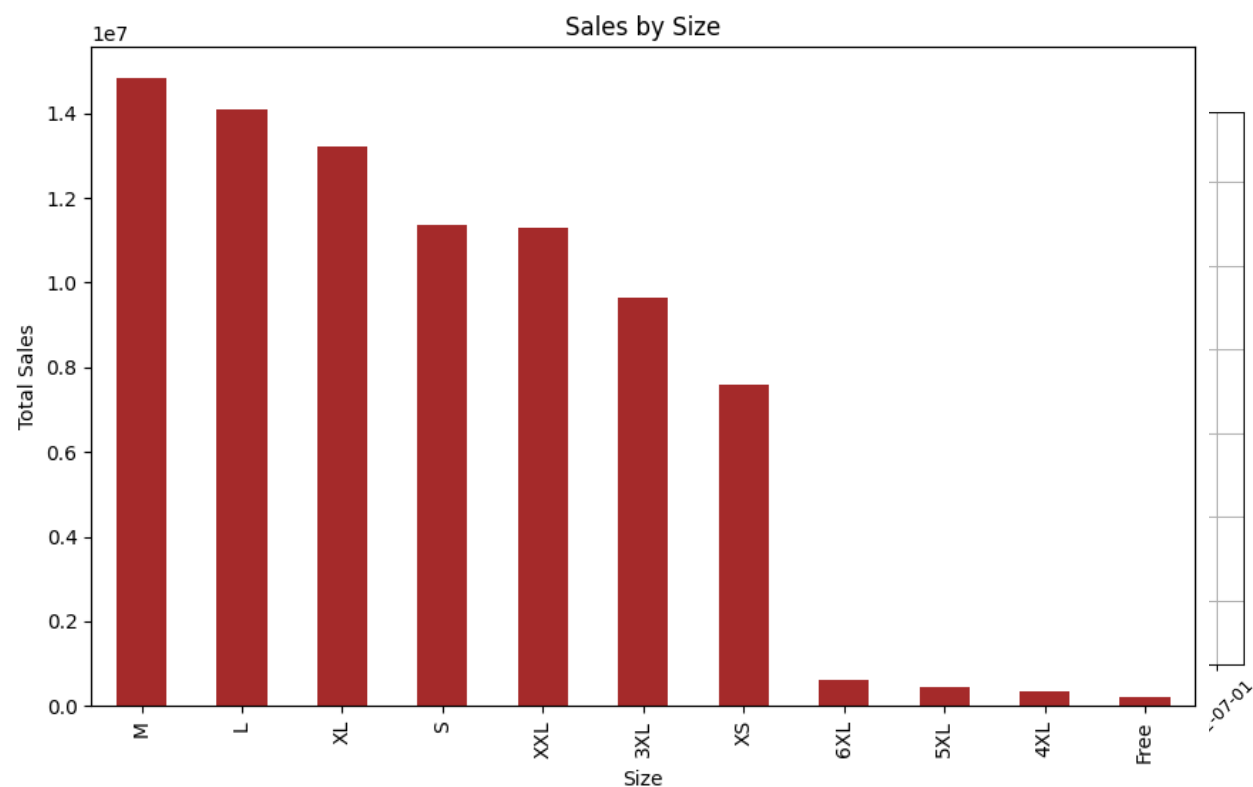
To further investigate relationships between categorical features and revenue, I encoded the 'Size', 'Category', and 'Status' columns using ordinal integer codes.

A correlation matrix was then generated to examine how these encoded features relate to 'Amount'.

This step provided a foundation for feature selection and multivariate analysis in the predictive modeling phase.

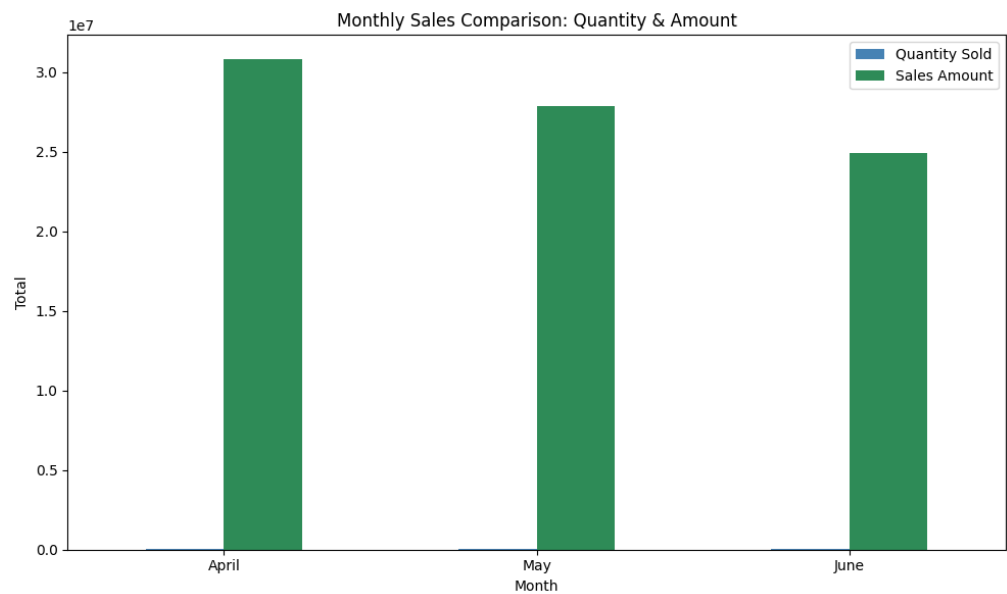
Data Visualization

1. Daily Sales Amount Trend



This line plot shows how the **sales amount fluctuated over time**, helping identify high-revenue and low-performance days.

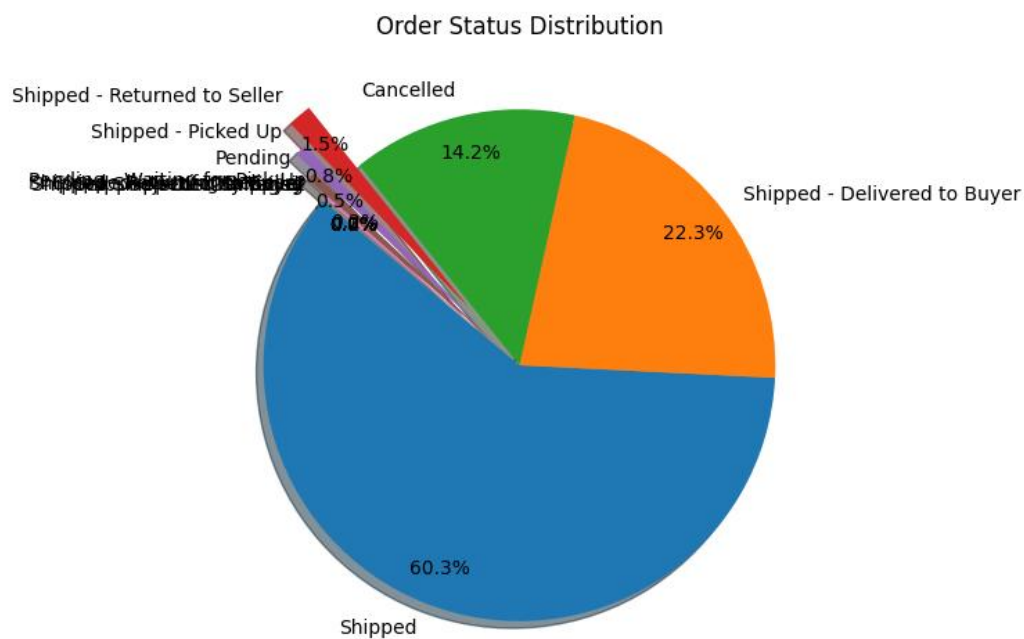
2. Monthly Sales Comparison (Quantity & Amount)



This bar chart allows comparison of **monthly performance** in terms of both **units sold** and **revenue generated**.

3. Sales by Size

This visualization identifies **which sizes drive the most revenue**, valuable for inventory and



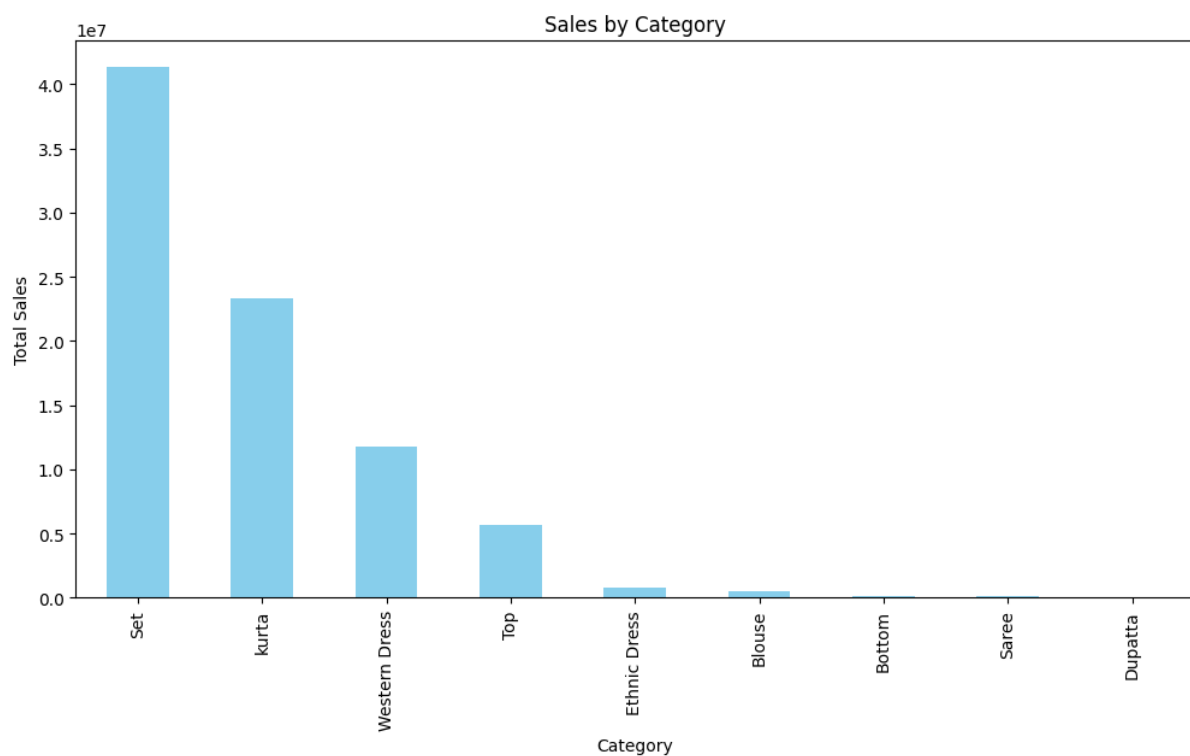
demand forecasting.

4. Sales by Category

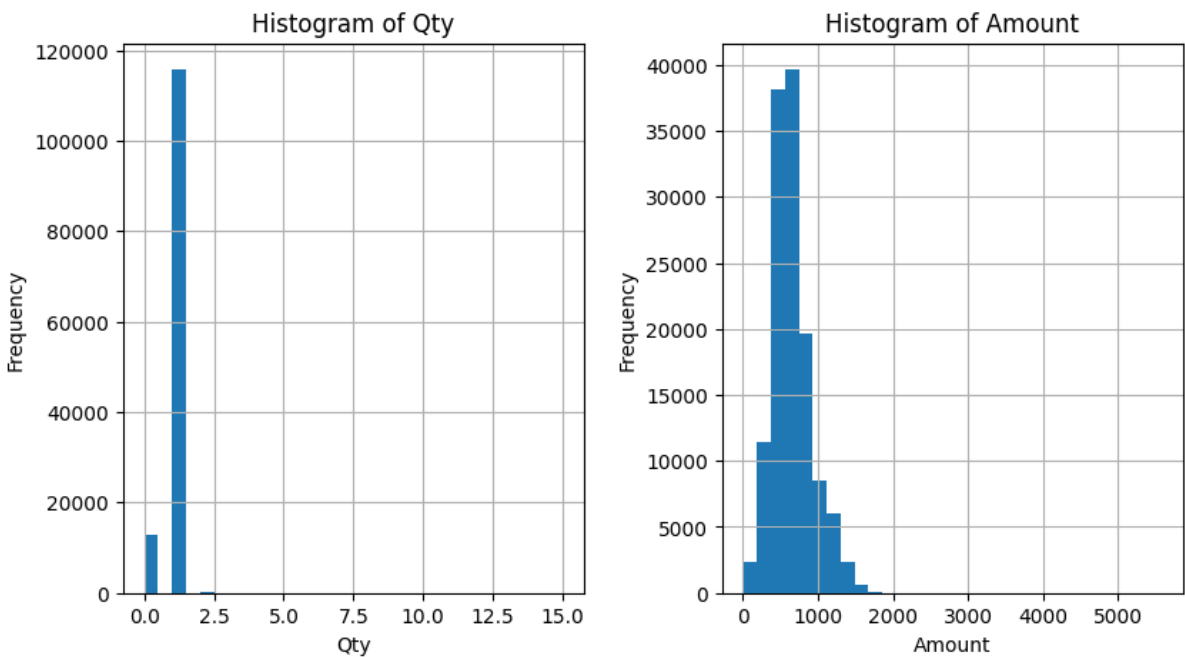
This helps understand **which product categories** are the top contributors to sales.

5. Order Status Distribution

The pie chart illustrates the **proportion of different order outcomes**, such as delivered, cancelled, and returned.



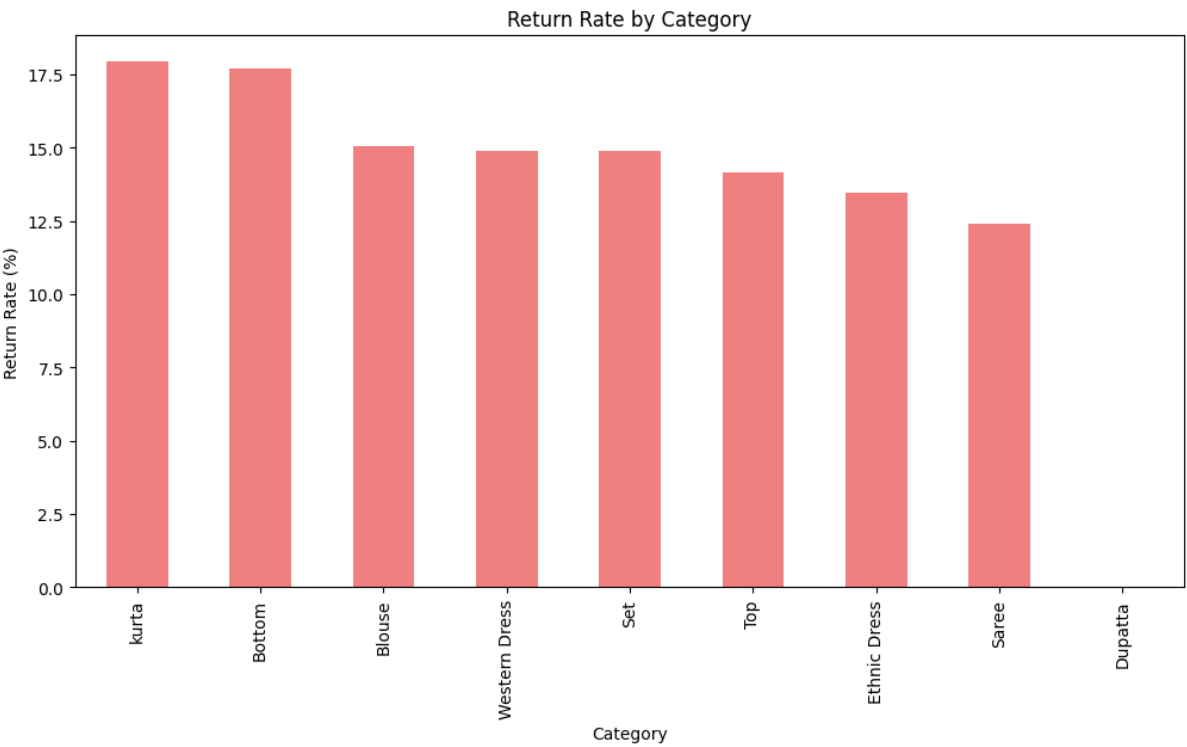
6. Return Rate by Category



This chart identifies **which categories have the highest return rates**, signaling potential issues with product quality or customer satisfaction.

7. Histograms of All Numeric Columns

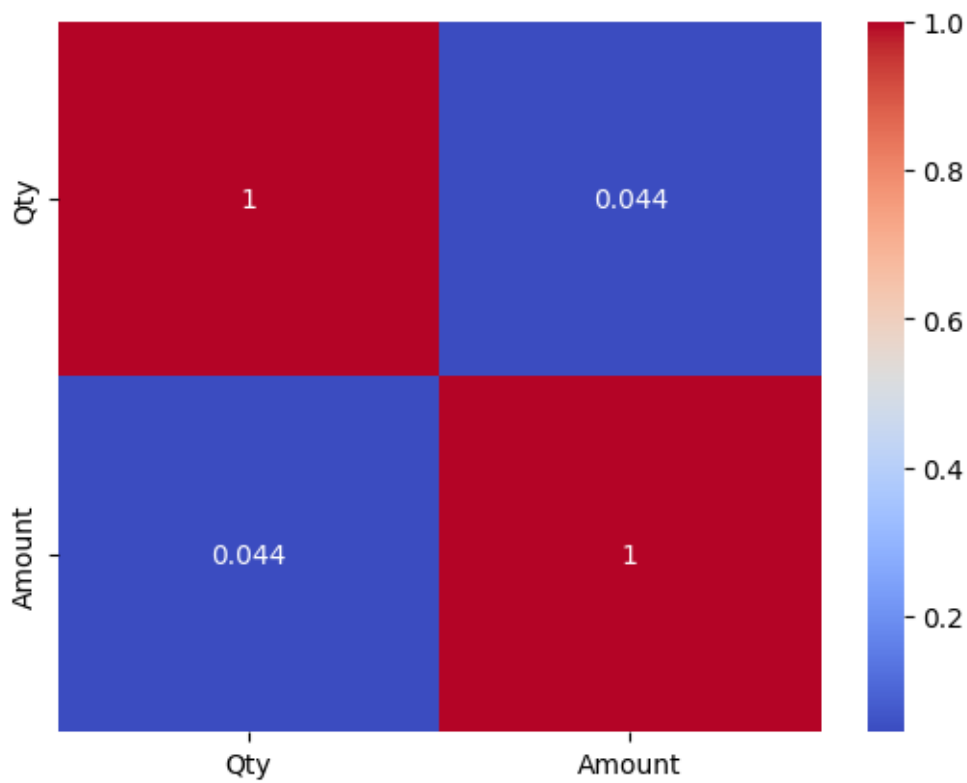
These histograms show the **distribution of numeric features**, highlighting data skewness or



outliers.

8. Correlation: Quantity vs Amount

A simple heatmap to assess if **quantity sold correlates with the revenue**.



9. Correlation Matrix: Encoded Categorical Features and Amount



This correlation matrix is useful for **feature engineering and modeling**, showing how encoded categorical variables relate to the Amount.

Tools

1. Matplotlib

- **Purpose:** Data visualization
- **Functions Used:**
 - `plt.figure()`, `plt.plot()`, `plt.bar()`, `plt.title()`
 - `plt.xlabel()`, `plt.ylabel()`, `plt.grid()`, `plt.xticks()`, `plt.tight_layout()`, `plt.show()`

2. Seaborn

- **Purpose:** Statistical data visualization
- **Functions Used:**
 - `sns.heatmap()` for correlation matrix visualization with annotations and color mapping.

3. Forecasting Model Development

To develop and evaluate accurate forecasting models using both statistical (ARIMA) and machine learning (Random Forest) techniques.

Approach

We performed the following:

- Selected suitable models (ARIMA, Random Forest).
- Trained and evaluated models using historical time series data.
- Optimized model parameters for better prediction performance.
- Tools used: Python with statsmodels, scikit-learn, pandas, and matplotlib.

A. ARIMA Model

1. Model Selection

- **Model Chosen:** ARIMA (AutoRegressive Integrated Moving Average)
- **Why ARIMA?:**
 - Suitable for **univariate time series** data.
 - Captures **trend, seasonality (with differencing)**, and **autocorrelation** in time series.
 - Works well with **stationary or differenced** data.
- **Parameters:**
 - order = (p,d,q):
 - p: Auto-regressive terms
 - d: Differencing to make the series stationary
 - q: Moving average terms
 - Default: (1,1,1) — but can be customized during optimization.

2. Model Training

- **Steps in Code:**
 - train_size = 80% → Train/Test split.
 - Training data: used to **fit** the ARIMA model using ARIMA(...).fit().
 - train() method encapsulates:
 - Preprocessing
 - ARIMA fitting
 - Metric calculation
- **Data Prepared With:**
 - Daily aggregation of sales.
 - Filling missing dates with 0.
 - Time series indexing with pd.date_range() and set_index().

3. Model Evaluation

- **Test Predictions:**
 - Forecasts are generated using .forecast() on the test set.
- **Metrics Calculated:**

- MAPE: Mean Absolute Percentage Error — measures average percent error.
- RMSE: Root Mean Squared Error — penalizes large errors.
- Forecast Accuracy: Derived from MAPE.
- Growth Rate: Measures trend in forecast.
- **Evaluation on Recent Data:**
 - During prediction, model also evaluates on most recent real data for dynamic metric feedback.

4. Model Optimization

- **Manual Parameter Setting:**
 - The model uses a fixed order=(1,1,1), but optimization can be added by:
 - **Grid Search** or **AIC/BIC minimization** over (p,d,q) values.
 - Use tools like `pmdarima.auto_arima()` for automated selection.
- **Realism Enhancements:**
 - Adds:
 - Controlled **random noise** (10% variation).
 - Slight **upward trend bias**.
 - **Adjusted confidence intervals** for more robust forecasts.
- **Error Handling:**
 - Try/except blocks prevent crashes during preprocessing and training.
 - Handles cases of insufficient data.

5. Tools Used

Purpose	Library/Tool
Data Handling	pandas, numpy
Time Series Modeling	statsmodels.tsa.ARIMA
Database Access	Custom Database class (from app.database)
Metrics & Evaluation	Manual functions using numpy
Error Handling	try/except, warnings filtered
Confidence Intervals	results.get_forecast(...).conf_int()
Date Handling	pd.date_range, pd.Timestamp

B.Random Forest

- **Model Selection**

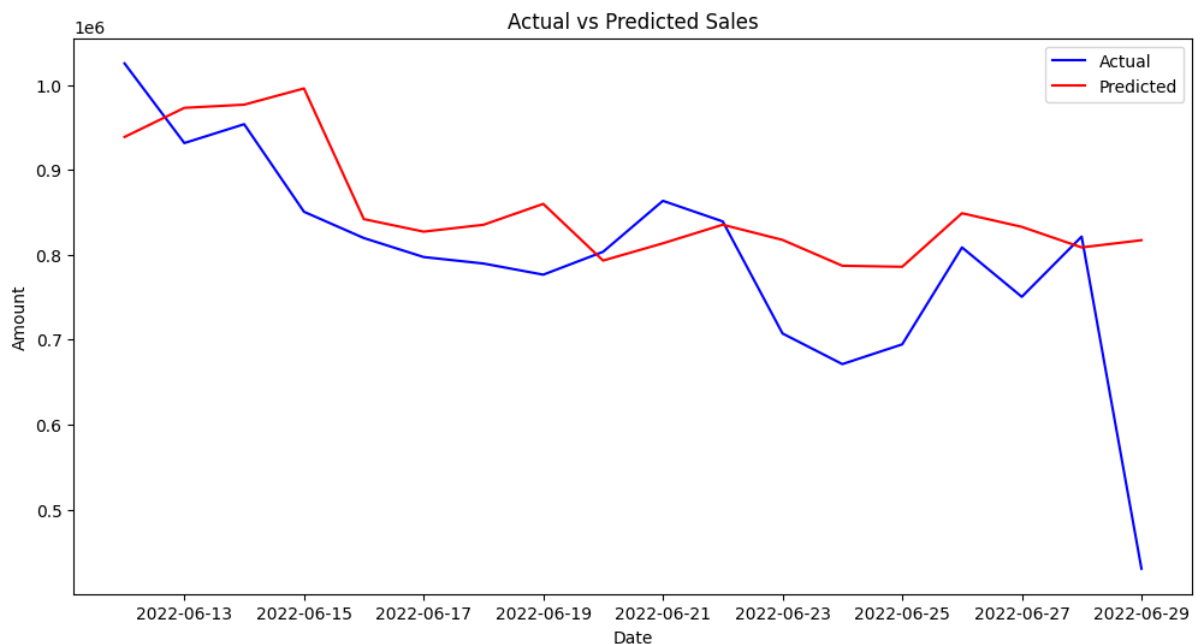
Random Forest was selected as a powerful ensemble machine learning model capable of handling nonlinear patterns and robustly predicting outcomes with multiple features. Unlike statistical models like ARIMA that assume a linear structure and require stationarity, Random Forest can learn from engineered features such as time lags and calendar-based inputs without requiring strict assumptions about the underlying data distribution. This makes it particularly suitable for capturing complex relationships in daily transaction or sales data.

- **Model Training**

The data was first grouped by date, and daily totals of the target variable Amount were computed. Several time-based features were engineered:

- **Lag Features:** Amount from the past 1, 2, and 3 days (lag_1, lag_2, lag_3).
- **Day of the Week:** Encoded numerically (0 = Monday, ..., 6 = Sunday).
- **Is Weekend:** A binary feature (1 if Saturday or Sunday, else 0).

The dataset was then split into training and test sets with chronological integrity (no shuffling) to preserve the time series order



- **Model Evaluation**

The model was evaluated using:

- **MAE (Mean Absolute Error):** Measures average magnitude of prediction errors. equals 56041.98
- **RMSE (Root Mean Squared Error):** Penalizes larger errors more heavily. equals 95552.73

4. MLOps and Development

1. A.MLops

- Why MLflow?

We chose MLflow because it provides:

- Experiment Tracking: We can log model parameters, metrics (MAE, RMSE, MAPE), and outputs.
- Model Registry: Enables version control for our ARIMA and Random Forest models.
- Reproducibility: Logs the exact code, data, and environment.
- Deployment Ready: Supports direct model deployment using REST APIs or containerized environments.

- How We Used MLflow

1. Tracking Experiments

Each training run of the ARIMA and Random Forest models was tracked using `mlflow.start_run()`:

- For ARIMA: order, training/test size, and time-series metrics were logged.
- For Random Forest: number of trees, lag features, and evaluation metrics were logged.

python

```
mlflow.log_param("model_type", "ARIMA")
```

```
mlflow.log_metric("rmse", rmse)
```

```
mlflow.log_metric("mape", mape)
```

2. Storing Models

Both models were saved in the MLflow model registry:

- This allows us to easily switch between model versions (e.g., ARIMA v1 → v2).
- Enables rollback in case of degraded performance.

3. Model Comparison

MLflow UI helped us compare performance (RMSE, MAE, MAPE) across:

- Model types (ARIMA vs. Random Forest)
- Data splits and feature engineering variations
- Hyperparameter tuning results

4. Version Control & Reproducibility

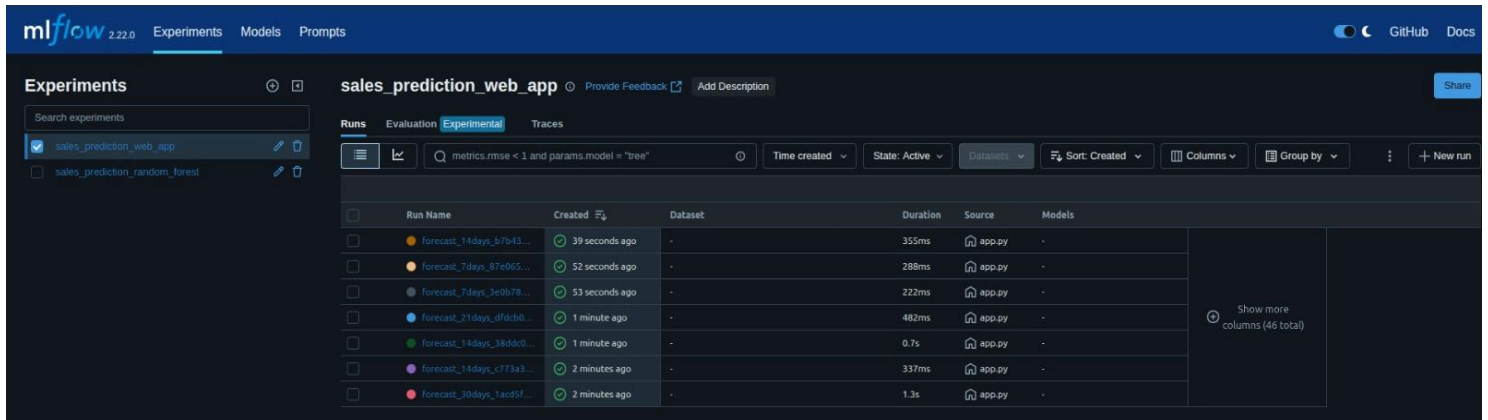
Each experiment was associated with:

- Git commit hash
- Dataset version
- Environment (Python, packages)

This guarantees complete traceability.

- 📦 MLflow Benefits in Our Project

Feature	Use in Project
Parameters Logging	Track order for ARIMA, number of estimators for RF
Metric Logging	Log RMSE, MAE, MAPE, Forecast Accuracy
Model Artifact	Save trained .pkl and .pmdarima models
Deployment Support	Use MLflow REST API or Docker container for future use



The screenshot shows the MLflow Experiments page for the experiment 'sales_prediction_web_app'. The 'Runs' tab is active, displaying a table of runs. The table has columns for Run Name, Created, Dataset, Duration, Source, and Models. There are 8 runs listed, each with a unique name and a creation time. A 'Show more columns (46 total)' button is visible on the right side of the table.

Run Name	Created	Dataset	Duration	Source	Models
Forecast_14days_b7b43...	39 seconds ago	-	355ms	app.py	-
Forecast_7days_87e065...	52 seconds ago	-	288ms	app.py	-
Forecast_7days_3e0b78...	53 seconds ago	-	222ms	app.py	-
Forecast_31days_dfbdb0...	1 minute ago	-	482ms	app.py	-
Forecast_14days_38ddc0...	1 minute ago	-	0.7s	app.py	-
Forecast_14days_c773a3...	2 minutes ago	-	337ms	app.py	-
Forecast_30days_1ad05f...	2 minutes ago	-	1.3s	app.py	-

B. Deployment

Why Flask?

We used Flask, a lightweight Python web framework, to:

- Build RESTful APIs for model interaction
- Deploy the forecasting logic to local or cloud environments
- Enable real-time predictions from frontend apps or other services

Flask Application Structure

app.py

Main application entry point:

- Provides API endpoints for:
 - /train: Trains ARIMA model
 - /predict: Returns next 7-day forecast
 - /metrics: Shows latest evaluation metrics

arima_forecaster.py

Encapsulates all logic for:

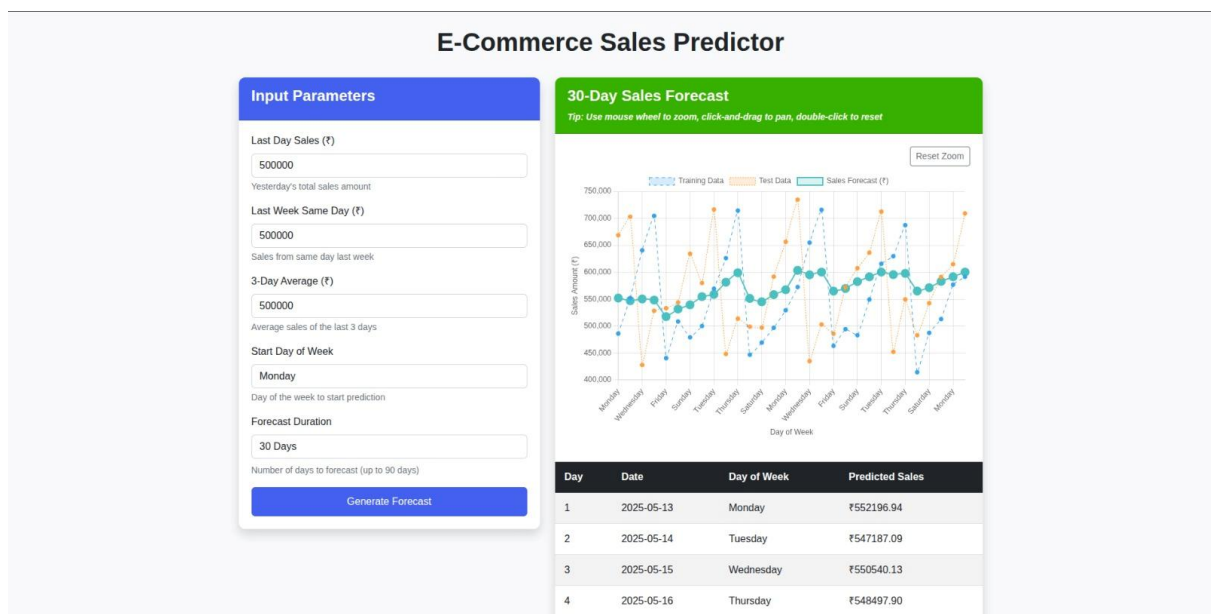
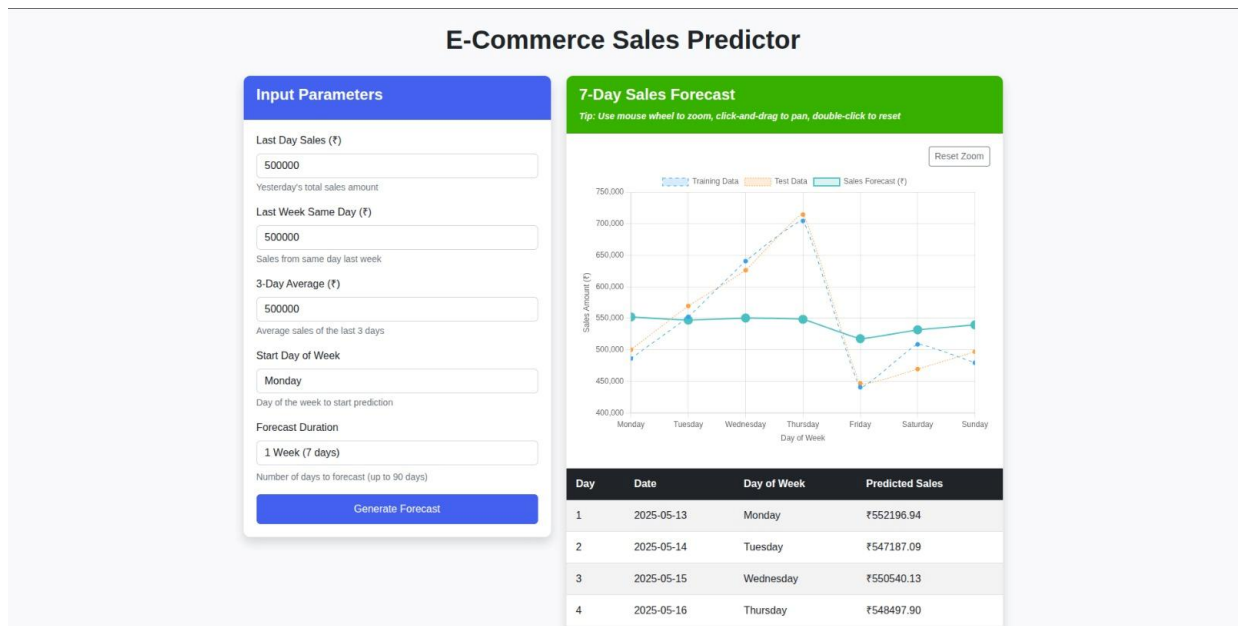
- Data loading & preprocessing
- Model training & forecasting
- Metric calculation (RMSE, MAPE, etc.)

This file is modular and testable.

database.py

Handles interaction with the database (mock or real):

- Pulls historical sales data
- Ensures daily frequency with complete date range



- API Architecture

Endpoint	Method	Description
/train	POST	Trains ARIMA model
/predict	GET	Returns forecast for next 7 days
/metrics	GET	Returns performance metrics

This allows front-end dashboards or mobile apps to consume our model outputs.

